EXPERIMENT NO. 09

1.1 AIM

To study and understand the working of a Distributed File System using Andrew File System (AFS) and to analyze how files are shared, accessed, and managed efficiently across multiple systems in a distributed environment.

1.2 OBJECTIVES

- To understand the concept of Distributed File Systems and their importance in distributed computing.
- To study the architecture and working of the Andrew File System (AFS).
- To understand file sharing and access mechanisms in AFS.
- To analyze caching and replication techniques used in AFS.
- To study consistency and synchronization mechanisms in distributed file systems.
- To understand security features such as authentication and access control in AFS.
- To evaluate the advantages and limitations of AFS in distributed environments.
- To explore real-world applications of distributed file systems.

1.3 SCOPE

1. Understanding Distributed File Systems and AFS Architecture

The scope of this experiment includes understanding the fundamental concepts of Distributed File Systems and how they enable multiple users to access and share files over a network. It focuses on studying the architecture of the Andrew File System (AFS), including its client-server model, file servers, and naming mechanisms. This helps in understanding how large-scale distributed environments manage data efficiently.

2. File Access, Sharing, and Caching Mechanisms

Another important scope is to study how files are accessed and shared among different users in AFS. The system uses caching techniques to improve performance by storing frequently accessed data locally. This reduces network load and enhances system efficiency. Understanding these mechanisms helps in designing systems that provide faster and more reliable file access.

3. Consistency, Synchronization, and Security

The scope also includes analyzing how AFS maintains data consistency across multiple clients and ensures proper synchronization when files are accessed or modified. Additionally, it focuses on security features such as authentication and access control, which ensure that only authorized users can access or modify files in the system.

4. Evaluation and Applications of AFS

The study further involves evaluating the advantages and limitations of AFS, such as scalability, performance, and fault tolerance. It also explores its applications in real-world scenarios like cloud storage systems, enterprise networks, and distributed computing environments. This helps in understanding the practical significance of distributed file systems in modern computing.

LITERATURE SUREVEY

Distributed File System

A distributed file system (DFS) is a networked architecture that allows multiple users and applications to access and manage files across various machines as if they were on a local storage device. Instead of storing data on a single server, a DFS spreads files across multiple locations, enhancing redundancy and reliability.

●This setup not only improves performance by enabling parallel access but also simplifies data sharing and collaboration among users.

●By abstracting the complexities of the underlying hardware, a distributed file system provides a seamless experience for file operations, making it easier to manage large volumes of data in a scalable manner.

Components of DFS

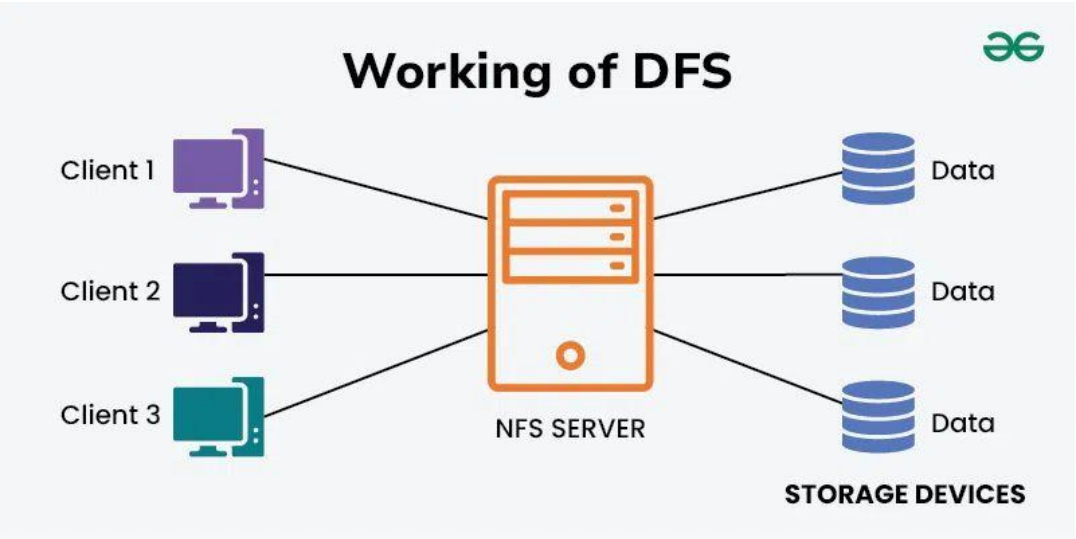
●Location Transparency: Location Transparency achieves through the namespace component.

●Redundancy: Redundancy is done through a file replication component.

In the case of failure and heavy load, these components together improve data availability by allowing the sharing of data in different locations to be logically grouped under one folder, which is known as the "DFS root". It is not necessary to use both the two components of DFS together, it is possible to use the namespace component without using the file replication component and it is perfectly possible to use the file replication component without using the namespace component between servers.

Distributed File System Replication
Early iterations of DFS made use of Microsoft's File Replication Service (FRS), which allowed for straightforward file replication between servers. The most recent iterations of the whole file are distributed to all servers by FRS, which recognises new or updated files. "DFS Replication" was developed by Windows Server 2003 R2 (DFSR). By only copying the portions of files that have changed

and minimising network traffic with data compression, it helps to improve FRS. Also, it provides users with flexible configuration options to manage network traffic on a configurable schedule.



Features of DFS
Transparency
1.Structure transparency: There is no need for the client to know about the number or locations of file servers and the storage devices. Multiple file servers should be provided for performance, adaptability, and dependability.

2.Access transparency: Both local and remote files should be accessible in the same manner. The file system should be automatically located on the accessed file and send it to the client's side.

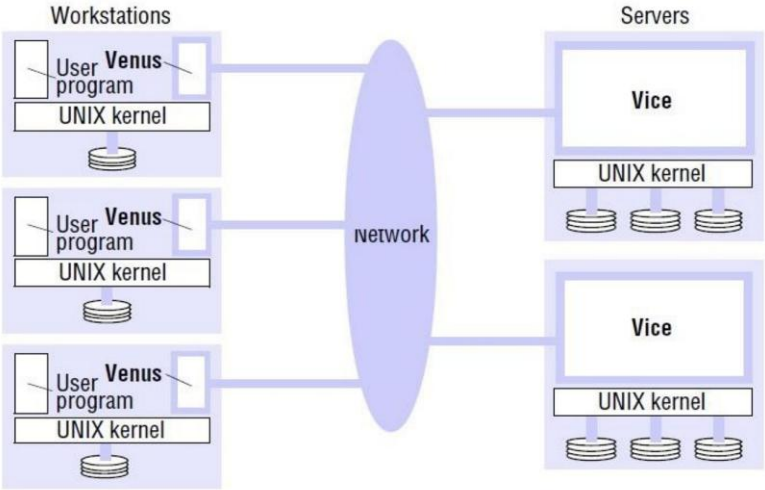
3.Naming transparency: There should not be any hint in the name of the file to the location of the file. Once a name is given to the file, it should not be changed during transferring from one node to another.

4.Replication transparency: If a file is copied on multiple nodes, both the copies of the file and their locations should be hidden from one node to another.User mobility: It will automatically bring the user's home directory to the node where the user logs in.

Performance: Performance is based on the average amount of time needed to convince the client requests. This time covers the CPU time + time taken to access secondary storage + network access time. It is advisable that the performance of the Distributed File System be similar to that of a centralized file system.
Simplicity and ease of use: The user interface of a file system should be simple and the number of commands in the file should be small.
High availability: A Distributed File System should be able to continue in case of any partial failures like a link failure, a node failure, or a storage drive crash.
A high authentic and adaptable distributed file system should have different and independent file servers for controlling different and independent storage devices.
Scalability: Since growing the network by adding new machines or joining two networks together is routine, the distributed system will inevitably grow over time. As a result, a good distributed file system should be built to scale quickly as the number of nodes and users in the system grows. Service should not be substantially disrupted as the number of nodes and users grows.
Data integrity: Multiple users frequently share a file system. The integrity of data saved in a shared file must be guaranteed by the file system. That is, concurrent access requests from many users who are competing for access to the same file must be correctly synchronized using a concurrency control method. Atomic transactions are a high-level concurrency management mechanism for data integrity that is frequently offered to users by a file system.
Security: A distributed file system should be secure so that its users may trust that their data will be kept private. To safeguard the information contained in the file system from unwanted & unauthorized access, security mechanisms must be implemented.

Andrew File System Architecture:

● Vice: The Andrew File System provides a homogeneous, location-transparent file namespace to all client workstations by utilizing a group of trustworthy servers known as
Vice. The Berkeley Software Distribution of the Unix operating system is used on both clients and servers. Each workstation's operating system intercepts file system calls and redirects them to a user-level process on that workstation.
● Venus: This mechanism, known as Venus, caches files from Vice and returns updated versions of those files to the servers from which they originated. Only when a file is opened or closed does Venus communicate with Vice; individual bytes of a file are read and written directly on the cached copy, skipping



This file system architecture was largely inspired by the need for scalability. To increase the number of clients a server can service, Venus performs as much work as possible rather than Vice. Vice only keeps the functionalities that are necessary for the file system's integrity, availability, and security. The servers are set up as a loose confederacy with little connectivity between them. The following are the server and client components used in AFS networks:

● Any computer that creates requests for AFS server files hosted on a network qualifies as a client.

● The file is saved in the client machine's local cache and shown to the user once a server responds and transmits a requested file.

● When a user visits the AFS, the client sends all modifications to the server via a call back mechanism. The client machine's local cache stores frequently used files for rapid access.

AFS implementation:

● Client processes communicate with a UNIX kernel via standard system calls.

● The kernel is tweaked significantly to identify references to Vice files in relevant activities and route requests to the workstation's Venus client process.

● If a volume is missing from this cache, Venus contacts any server it already has a connection with, asks for the location information, and enters it into the mapping cache. Venus makes a new connection to the server unless it already has one. The file or directory is then retrieved using this connection.

● Authentication and security need the establishing of a connection. A copy of the target file is made on the local disc when it is located and cached.

● Venus then returns to the kernel, which opens the cached copy and gives the client process the handle to it. Both AFS servers and clients use the UNIX file system as a low-level storage system. On the workstation's disc, the client cache is a local directory. This directory contains files with placeholder names for cache entries.

● Both Venus and server processes use the latter's modes to access UNIX files directly, avoiding the costly path-name-to-anode conversion method.

● When a file is deleted from the cache, Venus informs the relevant server that the call back for that file has been removed.

2.2 ALGORITHM (AFS – Distributed File System)

Algorithm: Andrew File System (AFS)

ALGORITHM AFS_File_Access

INPUT : File request F from client C
OUTPUT : File data returned to client

BEGIN

```
1. Client C requests file F

2. Check local cache:
   IF F exists in cache AND valid
       RETURN cached copy
   ENDIF

3. Resolve file location:
   Contact Name Server
   File_Server ← Lookup(F)

4. Request file from File_Server

5. File_Server processes request:
   IF F exists
       Send file data D to client
   ELSE
       Return error "File not found"
   ENDIF

6. Client receives file data D

7. Store file in local cache:
   Cache[F] ← D

8. Return file to user

END
```

Algorithm: AFS File Update (Write Operation)

```
ALGORITHM AFS_File_Update

INPUT  : File F, Updated data D
OUTPUT : Updated file stored on server

BEGIN

1. Client modifies file F locally

2. On file close:
   Send updated file D to File_Server

3. File_Server updates file:
   F ← D

4. Invalidate other cached copies:
   For each client Ci
       Mark CacheCi[F] as invalid
   End For

5. Acknowledge update to client

END
```

IMPLEMENTATION

2.3 Programming Language Used
Python Programming Language

The Distributed File System experiment using Andrew File System (AFS) is implemented in the Python programming language. Python is chosen because of its simple syntax, readability, and ability to simulate complex system behavior using object-oriented programming concepts.
In this implementation, the AFS system is simulated using classes, methods, and dictionaries to represent the file server, client cache, and file operations. Although actual AFS systems work over networks, this program demonstrates the core concepts such as file access, caching, and update mechanisms within a single execution environment.

2.3.1 Libraries Used
This implementation does not require any external libraries. It uses only built-in features of Python.
1) Built-in Python Functions
Python provides default functions such as:
●print() – used to display output

●input() – (optional) for user interaction

These functions are sufficient for simulating the AFS behavior.

2.3.2 Concepts Used
1) Class
A class is used to define the structure of the system components.
Syntax:
```
class ClassName:
  pass
```
Used to create:
●FileServer

●Client

2) Dictionary (Key-Value Storage)
Used to store files and their contents.
Syntax:
```
dict_name = { "key": "value" }
```
Usage:
●Stores file names and file data

●Simulates server storage

2.3.3 Classes and Methods Used
1) FileServer Class
This class represents the central server in the AFS system.
Methods:
a) init()
Purpose:
Initializes the file server with predefined files.
Syntax:
```
def __init__(self):
```
Usage:
●Creates a dictionary of files

●Acts as server storage

b) read_file(filename)
Purpose:
Handles file read requests from the client.
Syntax:
```
def read_file(self, filename):
```
Working:
●Checks if file exists in server

●If yes → returns file content

●If no → returns error

c) write_file(filename, data)
Purpose:
Handles file update requests.
Syntax:
def write_file(self, filename, data):
Working:
●Updates or creates file in server

●Stores new data

2) Client Class
This class represents the user/client system.
Methods:
a) init(server)
Purpose:
Initializes the client with a reference to the server.
Syntax:
def __init__(self, server):
Working:
●Creates local cache

●Connects client to server

b) read(filename)
Purpose:
Handles file read operation from client side.
Syntax:
def read(self, filename):
Working:
1.Checks if file is present in local cache

2.If present → returns cached copy

3.If not → requests file from server

4.Stores received file in cache

c) write(filename, data)
Purpose:
Handles file update operation.
Syntax:
def write(self, filename, data):
Working:
1.Sends updated data to server

2.Updates local cache

3.Simulates AFS "write-on-close" behavior

2.3.4 Variables Used
1) Server Variables
●files → Dictionary storing all server files

Example:
self.files = {
  "file1.txt": "Hello",
  "file2.txt": "Data"
}

2) Client Variables
●cache → Stores locally cached files

●server → Reference to FileServer object

3) Temporary Variables
●filename → Name of file

●data → File content

 # AFS Simulation in Python

```
class FileServer:
    def __init__(self):
        self.files = {
            "file1.txt": "Hello from AFS!",
            "file2.txt": "Distributed Systems Lab"
        }

    def read_file(self, filename):
        if filename in self.files:
            print("Server: Sending file ->", filename)
            return self.files[filename]
        else:
            return None

    def write_file(self, filename, data):
        self.files[filename] = data
        print("Server: File updated ->", filename)


class Client:
    def __init__(self, server):
        self.cache = {}
        self.server = server

    def read(self, filename):
        # Check cache
        if filename in self.cache:
            print("Client: Reading from cache ->", filename)
            return self.cache[filename]

        # Fetch from server
        print("Client: Requesting from server ->", filename)
        data = self.server.read_file(filename)

        if data:
            self.cache[filename] = data  # Cache it
            return data
        else:
            return "File not found"
```

```
    def write(self, filename, data):
        print("Client: Updating file ->", filename)
        self.server.write_file(filename, data)

        # Update local cache
        self.cache[filename] = data


# Create server
server = FileServer()

# Create client
client = Client(server)

# READ operation
print("\n--- READ OPERATION ---")
print(client.read("file1.txt"))

# READ again (from cache)
print("\n--- READ FROM CACHE ---")
print(client.read("file1.txt"))

# WRITE operation
print("\n--- WRITE OPERATION ---")
client.write("file1.txt", "Updated AFS Content")

# READ updated file
print("\n--- READ UPDATED FILE ---")
print(client.read("file1.txt"))
```

Command:- run
Platform:- Google Colab

```
[1]
client = Client(server)

# READ operation
print("\n--- READ OPERATION ---")
print(client.read("file1.txt"))

# READ again (from cache)
print("\n--- READ FROM CACHE ---")
print(client.read("file1.txt"))

# WRITE operation
print("\n--- WRITE OPERATION ---")
client.write("file1.txt", "Updated AFS Content")

# READ updated file
print("\n--- READ UPDATED FILE ---")
print(client.read("file1.txt"))
```

```
--- READ OPERATION ---
Client: Requesting from server -> file1.txt
Server: Sending file -> file1.txt
Hello from AFS!

--- READ FROM CACHE ---
Client: Reading from cache -> file1.txt
Hello from AFS!

--- WRITE OPERATION ---
Client: Updating file -> file1.txt
Server: File updated -> file1.txt

--- READ UPDATED FILE ---
Client: Reading from cache -> file1.txt
Updated AFS Content
```

Conclusion:-

A Distributed File System (DFS) provides a unified and efficient way to store, access, and manage data across multiple machines as if it were a single system. It enhances data availability, scalability, and fault tolerance by distributing files across different nodes, ensuring that system failure in one location does not lead to complete data loss.
DFS enables concurrent access by multiple users, improves performance through data replication and load balancing, and supports seamless sharing of resources in large-scale environments such as cloud computing and enterprise networks. However, it also introduces challenges like security, consistency management, and network dependency, which must be carefully handled.
In conclusion, a Distributed File System is a powerful solution for modern data management, offering flexibility and reliability, but requiring well-designed mechanisms to maintain data integrity, synchronization, and security across distributed environments.

Sign and Remarks:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature